

Flexbone OCR

Implementation and Infrastructure Explanation

OCR service, upload validation, system architecture, and deployment strategy

Overall introduction

Flexbone OCR is a small, stateless API I built to accept an image and return the text found in it. Google Cloud Vision handles the OCR, FastAPI provides the API, and Google Cloud Run hosts it. The service supports both single-image and batch requests without saving uploaded images or OCR text.

Live links

Browser tester	https://ocr.lalbaba.top
API base URL	https://api.ocr.lalbaba.top
API documentation	https://api.ocr.lalbaba.top/docs
OpenAPI schema	https://api.ocr.lalbaba.top/openapi.json

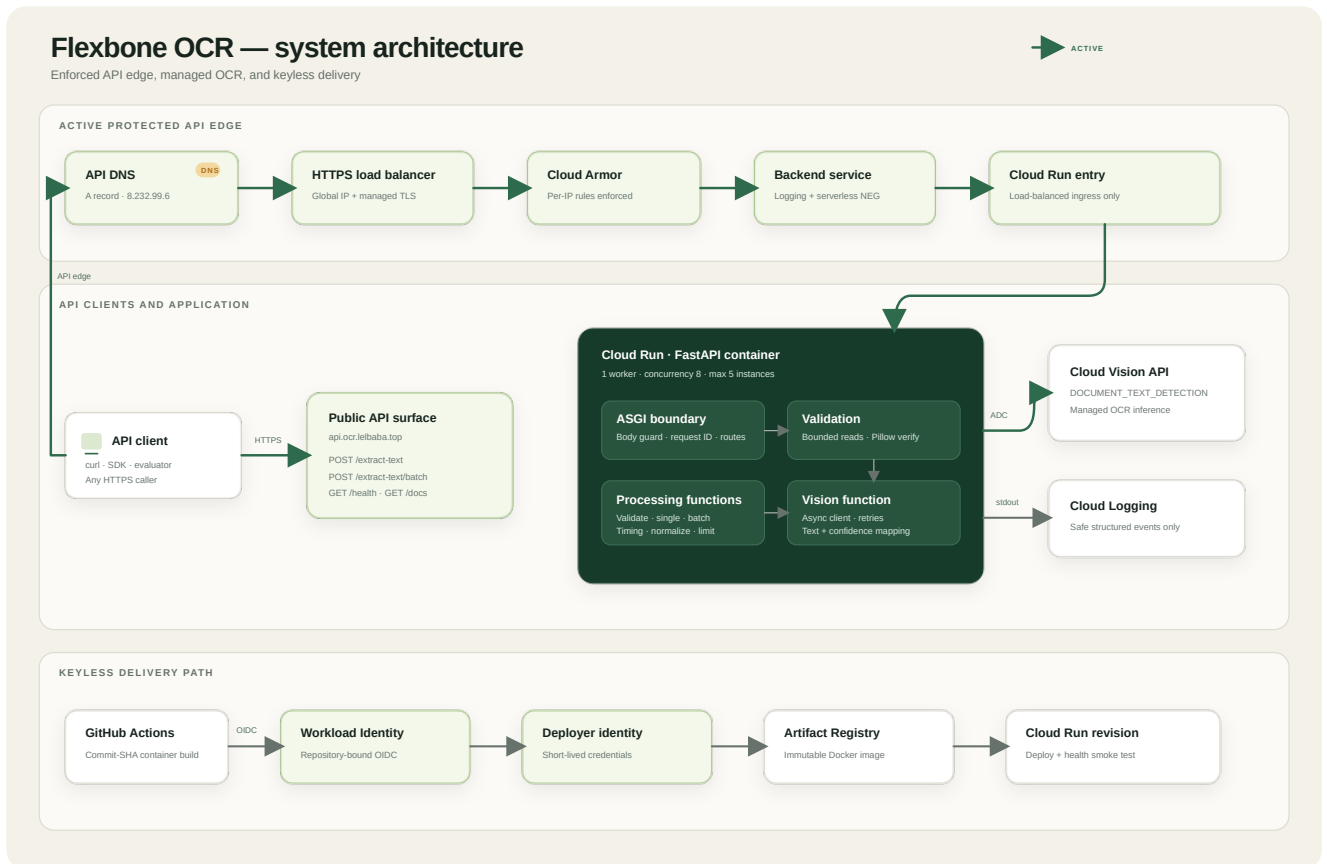
1. Technology stack

The stack is intentionally simple and mostly managed. That lets me focus on safe image handling and reliable OCR instead of maintaining servers or training an OCR model. The Python API is packaged as a Docker image and deployed to Google Cloud, with automated checks verifying the code and container before release.

Python 3.12	Python powers the backend because it has clear async support and mature libraries for image processing and Google Cloud.
FastAPI and Uvicorn	FastAPI defines the HTTP endpoints and response models. Uvicorn runs the FastAPI application inside the container.
Google Cloud Vision API	OCR is handled by the DOCUMENT_TEXT_DETECTION feature. It works well for document-style text, handwriting, rotated images, and degraded images without requiring me to host a model.
Pillow	Pillow opens and verifies the real image content, so validation does not depend on an untrusted filename, extension, or MIME type.
Pydantic Settings	Typed settings keep limits, timeouts, the allowed browser origin, and other configuration in one place. Environment variables use the OCR_ prefix.
Docker	The backend is packaged as a reproducible, non-root container. The same Dockerfile is built in CI and deployed to Cloud Run.
uv and uv.lock	uv installs the dependencies, while the lock file keeps local, CI, and container environments consistent.
pytest and HTTPX	Together, they test API behavior, validation, error responses, concurrency, retries, and integration with the deployed service.
Ruff and mypy	Ruff checks formatting and code quality, while strict mypy checks type safety.
GitHub Actions	GitHub Actions runs continuous integration and the controlled production deployment workflow.

2. Overall system architecture

All API traffic goes through the protected Google Cloud edge before it reaches the FastAPI container. The container validates the upload, sends the image bytes to Google Cloud Vision, and returns a JSON response. There is no other route in: the container itself refuses any request that does not arrive through the load balancer.



Flexbone OCR system architecture

An API client sends an image to the API domain over HTTPS. The global HTTPS load balancer receives the request, Cloud Armor applies the per-IP rate limits, and the request reaches Cloud Run through a serverless network endpoint group. Inside the container, FastAPI validates the upload and calls Google Cloud Vision, then returns the JSON response back along the same path. Safe operational events go to Cloud Logging, but the uploaded image and the extracted text are never logged.

2.1. The FastAPI codebase

I kept the backend deliberately small, with one clear responsibility for each file. This makes the request flow easy to follow and test without adding a large framework or unnecessary abstraction.

app.py	This is the entry point of the API. It defines the health, single-image, and batch endpoints; starts the reusable Vision client; maps errors into one JSON format; and writes safe completion logs.
middleware.py	This is where the total HTTP request-body limit is enforced, including for streamed requests. It also handles request IDs and adds timing headers to every response.
processing.py	This module handles the main processing flow: reading bounded chunks, validating with Pillow, applying image and batch limits, controlling concurrency, enforcing deadlines, optionally normalizing text, and preserving result order.
vision.py	All communication with Google Cloud Vision lives here, including attempt timeouts, retries for temporary failures, text extraction, and confidence calculation.
models.py	This file defines the public response shapes and the safe error table. Expected failures use a small <code>AppError</code> code instead of exposing internal Google or Python errors.
config.py	All typed limits and timeouts live here, giving me one place to review or change the service's operational boundaries.

Important implementation features

File size limits	Each image is limited to 10 MiB. The middleware also rejects an oversized HTTP body early, and streamed chunks are counted so a client cannot bypass the limit.
Image validation	The accepted formats are JPEG, PNG, and GIF. Pillow checks the decoded format, verifies the file, and rejects corruption or more than 40 million pixels. For a GIF, only the first frame is processed.
Safe upload handling	Each upload is read in 64 KiB chunks and stops as soon as it crosses the limit. Upload handles close on both success and failure, and the image is never saved permanently.

2.2. API features and safeguards

Single-image OCR	The /extract-text endpoint accepts one image and returns the raw OCR text, confidence score, processing time, and optional safe metadata or normalized text.
Batch OCR	The batch endpoint accepts one to five images. Each image can be 10 MiB, but validated image data is limited to 25 MiB for the full batch.
Batch concurrency	A batch can make at most two Vision calls at the same time. This controls provider usage and memory while preserving the original result order.
Timeouts and retries	A request has a 50-second application deadline, shorter than Cloud Run's 60-second limit. A Vision attempt has a 20-second deadline, and temporary failures can be retried twice.
Consistent errors	Validation failures, unknown routes, provider errors, timeouts, and unexpected failures use one safe JSON error structure. Internal exception details stay in logs.
Privacy	Image bytes, filenames, EXIF data, and OCR text are never retained or logged. Logs contain only safe operational details such as format, byte count, status, timing, and retry count.
CORS	The API allows browser requests from one configured origin only, and credentials are disabled because the API has no authentication. This keeps browser access narrow, while command-line and server-side API clients are unaffected.
Request IDs	Every response includes an X-Request-ID, which lets me connect an API report with Cloud Run logs without recording sensitive content.
Rate limiting	Google Cloud Armor applies per-IP rate limits at the load balancer before requests reach FastAPI: ten single-image requests and two batch requests per minute per IP address, with anything above that answered by the edge as a 429. This is an infrastructure safeguard rather than application middleware, so it keeps working even if the application is busy.
Scaling and cost limits	Cloud Run can scale from zero to a maximum of five instances. Each instance uses one CPU, 512 MiB of memory, and concurrency of eight.

The main design idea is defense in depth: Cloud Armor controls request frequency, middleware controls transport size, Pillow controls decoded image safety, processing code controls concurrency and deadlines, and Vision performs the OCR.

2.3. Infrastructure components

Unless I note otherwise, every component below is a Google Cloud (GCP) service inside the same project.

Public DNS records	The authoritative DNS records live at my domain registrar rather than in Cloud DNS. The API name is an A record pointing at the load balancer's global static IP address.
Certificate Manager	Certificate Manager issues and automatically renews the TLS certificate that the load balancer presents for the API domain.
HTTPS load balancer	A global external Application Load Balancer is the only public entry point for the API. It terminates HTTPS and forwards permitted traffic toward Cloud Run.
Cloud Armor	Cloud Armor enforces the per-IP rate limits at the edge, so unwanted traffic is rejected before it can consume application or Vision resources.
Serverless NEG	A serverless network endpoint group is the supported bridge from the load balancer to Cloud Run. Its backend service is what attaches Cloud Armor and request logging to the route.
Cloud Run	Cloud Run runs the non-root FastAPI container. It manages revisions, automatic scaling, the runtime service identity, and rollback.
Cloud Vision API	Vision performs DOCUMENT_TEXT_DETECTION on the validated image bytes and returns the text and per-word confidence to the backend.
Artifact Registry	Artifact Registry is GCP's managed container-image registry. Every production image is stored there under its immutable Git commit SHA.
IAM service accounts	Two separate service accounts split deployment from runtime access. The deployer can only push and release images, while the runtime identity can only reach the Google services the application actually needs.
Workload Identity Federation	Workload Identity Federation gives GitHub Actions short-lived Google credentials, so no permanent service-account key is ever stored in GitHub.
Cloud Logging	Cloud Logging collects safe events from the application, Cloud Run, the load balancer, and Cloud Armor for troubleshooting and request tracing.
GitHub Actions	GitHub Actions, on GitHub rather than GCP, runs the quality checks and the production workflow that builds, publishes, deploys, and verifies the backend.

3. CI/CD and deployment strategy

My release process combines automatic continuous integration with a controlled production release. Every push and pull request runs CI, while production deployment is started manually from the main branch through GitHub's production environment. Because I decide when to release, this is continuous delivery rather than fully automatic continuous deployment.

Continuous integration

1. GitHub Actions checks out the exact commit and installs the locked dependencies with uv.
2. It checks the lock file, formatting, lint rules, and strict Python types.
3. It runs the offline pytest suite with an 85 percent application coverage requirement.
4. It builds the Dockerfile and confirms the application can be imported inside the production image.

Production delivery

1. When I trigger the workflow on main, GitHub creates a short-lived identity token. Google verifies that it came from my repository and temporarily authorizes the deployment service account.
2. The workflow builds the Docker image and tags it with the Git commit SHA, so I can trace every image to one exact source revision.
3. It pushes that immutable image to Google Cloud Artifact Registry.
4. It creates a new Cloud Run revision with the protected ingress, scaling limits, timeouts, and runtime service account.
5. Cloud Run pulls the selected image from Artifact Registry. Public traffic continues to enter through the load balancer and Cloud Armor.
6. The workflow calls the public health endpoint and then runs real OCR integration tests through the production API domain.

How deployment connects to the infrastructure

GitHub Actions is the delivery controller, Artifact Registry is the handoff point for the built image, and Cloud Run is the runtime. The load balancer and Cloud Armor remain in front of every new revision because the deployment command preserves load-balanced-only ingress and disables the direct Cloud Run URL.

Rollback

Every deployment creates a new Cloud Run revision and keeps earlier revisions available. If a release is faulty, I can route all traffic back to a known-good revision without rebuilding it. The current workflow verifies the new release after deployment, but rollback is a deliberate operational action rather than an automatic workflow step.